Analysis of Agentic AI Platforms for Autonomous Multi-Agent Ecosystems

Ikram Aziz
Master's Student in Internet of Things (IoT), EURECOM
Supervisor: Prof. Jérôme Härri

June 2026

Abstract

Future autonomous transportation systems will not be built as a single monolithic program, but as ecosystems of many independent software *agents* — representing vehicles, traffic-management systems, charging stations, weather services and more — that must cooperate despite being built by different vendors, on different technologies, with different internal data models. This report surveys the current state of the art of the platforms, protocols and standards proposed to make such cooperation possible: **Agntcy** (with its CoffeeAGNTCY reference applications, *Corto* and *Lungo*), **OpenSlice**, the **W3C/A2A agent protocols**, **MCP**, **GITER**, and **ontology-based** semantic approaches. For each technology the report defines the relevant technical vocabulary, describes the underlying protocols and standards in detail, and provides architecture diagrams of how the system operates end to end. The report is deliberately descriptive: it characterises each technology on its own terms rather than ranking them. It closes with a neutral gap analysis observing that, across the surveyed technologies, the communication and interoperability layers are well covered while the semantic-reasoning layer remains comparatively underdeveloped.

Contents

1 Introduction and Problem Statement **2**
1.1 Context: cooperation between heterogeneous agents . . . 2
1.2 Research question . . . 3
1.3 Objectives . . . 3
1.4 What every agent ecosystem needs (analysis framework) . . . 3

2 Background: Protocols and Standards **3**
2.1 Communication and data-exchange standards . . . 3
2.2 Agent-to-agent communication: the A2A protocol . . . 4
2.3 Tool access: the Model Context Protocol (MCP) . . . 5
2.4 Secure transport: SLIM . . . 5
2.5 Agent description and discovery: OASF and the Agent Directory . . . 5
2.6 Identity and authorization: TBAC . . . 6
2.7 Telecom / service-orchestration standards (used by OpenSlice) . . . 6
2.8 Semantic web standards (used by ontology approaches) . . . 7

3 Agntcy **7**
3.1 Overview . . . 7
3.2 How Agntcy works, end to end . . . 7
3.3 The CoffeeAGNTCY reference applications . . . 8
3.3.1 Corto — the minimal two-agent flow . . . 8
3.3.2 Lungo — the larger multi-agent flow . . . 9
3.3.3 Corto versus Lungo . . . 11
3.4 Agntcy in the six-layer framework . . . 12

4 OpenSlice **12**
4.1 Overview . . . 12
4.2 How OpenSlice works, end to end . . . 12
4.3 Deployment status . . . 13
4.4 OpenSlice in the six-layer framework . . . 14

5 W3C Agent Protocol, MCP and GITER **14**
5.1 W3C / A2A agent protocols . . . 14
5.2 MCP . . . 15
5.3 GITER . . . 15
5.4 Common observation . . . 15

6 Semantic Reasoning Approaches **15**
6.1 Communication is not understanding . . . 15
6.2 Ontologies as shared meaning . . . 15
6.3 Do agent ecosystems require an LLM? . . . 15

7 Comparative Discussion **16**

8 Gap Analysis and Conclusion **16**
8.1 Findings . . . 16
8.2 Gap . . . 17
8.3 Summary . . . 17

Appendix: Index of defined terms **17**

A note on terminology. Throughout this report, technical terms are defined in shaded boxes the first time they appear. Each definition ends with a footnote linking to a public reference (typically Wikipedia or the official specification) for readers who wish to go deeper. The definitions are intended for a reader who is not previously familiar with multi-agent systems.

1 Introduction and Problem Statement

1.1 Context: cooperation between heterogeneous agents

Autonomous transportation increasingly relies on coordination between many independent decision-making components. A connected vehicle does not operate in isolation: it interacts with traffic-management infrastructure, charging or refuelling stations, weather and road-condition services, and cloud or edge services. Each such component can be modelled as an *autonomous agent*.

Autonomous agent

A software entity that perceives its environment, makes decisions according to its own goals, and acts — with little or no human intervention. In a multi-agent system, several such agents interact to achieve individual or collective objectives.[a]

[a] https://en.wikipedia.org/wiki/Software_agent

Multi-agent system (MAS)

A system composed of multiple interacting agents. Cooperation, coordination and (sometimes) competition between the agents produce the overall behaviour of the system.[a]

[a] https://en.wikipedia.org/wiki/Multi-agent_system

The difficulty is that these agents are *heterogeneous*. They come from different vendors, are implemented with different technologies, and represent knowledge in different ways. An agent that emits the token `AV` may mean "autonomous vehicle", while a receiving agent may have no entry for that token at all. Making heterogeneous agents communicate, cooperate and — ideally — reason together is the central problem this report addresses.

1.2 Research question

> *How can autonomous agents communicate, cooperate, and reason together, and what is the current state of the art of the platforms, protocols and standards that aim to make this possible?*

1.3 Objectives

1. Study existing agentic AI platforms.
2. Describe in detail the communication protocols they use.
3. Describe the interoperability mechanisms they provide.
4. Survey ontology-based reasoning approaches for shared meaning.

1.4 What every agent ecosystem needs (analysis framework)

To describe each technology consistently, this report uses a simple six-layer framework. It is a *descriptive* lens, not a scoring system: it is used only to say which concerns a given technology was designed to address.

Layer	Concern
Communication	Agents exchange messages reliably.
Discovery	Agents find and address one another.
Orchestration	Tasks and workflows are coordinated across agents.
Security	Identity, authentication, authorization.
Semantics	Shared meaning of the exchanged data.
Reasoning	Inference and decision-making over that data.

Table 1: The six-layer framework used to describe each technology.

2 Background: Protocols and Standards

This section defines the protocols and standards that recur throughout the report, before the technologies that use them are described. Readers familiar with these may skip to Section 3.

2.1 Communication and data-exchange standards

HTTP / HTTPS

The HyperText Transfer Protocol is the request–response protocol of the web; HTTPS is HTTP over an encrypted TLS channel. Many agent protocols run on top of HTTP(S).[a]

[a] https://en.wikipedia.org/wiki/HTTP

JSON / JSON-RPC 2.0

JSON (JavaScript Object Notation) is a lightweight text format for structured data. JSON-RPC 2.0 is a remote-procedure-call convention that encodes a method name and parameters as a JSON object, so one program can invoke a function in another over the network.[a]

[a] https://en.wikipedia.org/wiki/JSON-RPC

gRPC

A high-performance remote-procedure-call framework that uses HTTP/2 for transport and Protocol Buffers for serialization. It natively supports streaming in both directions.[a]

[a] https://en.wikipedia.org/wiki/GRPC

Server-Sent Events (SSE)

A standard that lets a server push a continuous stream of updates to a client over a single HTTP connection. Used by agent protocols to stream partial or progressive results.[a]

[a] https://en.wikipedia.org/wiki/Server-sent_events

Publish/subscribe (pub/sub)

A messaging pattern in which senders (publishers) do not address messages to specific receivers; instead messages are published to topics, and any interested party (subscriber) receives them. This decouples senders from receivers.[a]

[a] https://en.wikipedia.org/wiki/Publish%E2%80%93subscribe_pattern

2.2 Agent-to-agent communication: the A2A protocol

A2A (Agent-to-Agent) protocol

An open standard that defines how independent agents discover each other, delegate tasks and exchange data. A2A uses JSON-RPC 2.0 over HTTP(S), with Server-Sent Events for streaming and optional gRPC bindings. It was donated to the Linux Foundation in 2025 and reached version 1.0 in early 2026.[a]

[a] https://a2a-protocol.org/latest/specification/

A2A defines *what agents say* to one another. Its design rests on a few ideas relevant to this report:

- **Agent Card.** A machine-readable description of an agent — its identity, skills, capabilities, interfaces and security requirements — typically published at the well-known URL `/.well-known/agent-card.json`. This is how an agent advertises what it can do, enabling *discovery*.
- **Client / server roles.** An A2A *client* sends a request; an A2A *server* receives it and responds. A single agent is often both.
- **Interaction modes.** Synchronous request/response, streaming (via SSE), and asynchronous push notifications for long-running tasks.
- **Task lifecycle.** A task is an object with a defined lifecycle (submitted, working, completed, failed, . . .), so agents can coordinate multi-step work.

2.3 Tool access: the Model Context Protocol (MCP)

MCP (Model Context Protocol)

An open standard for connecting an agent (or the model behind it) to external *tools* and *data sources*. An *MCP client* (inside the agent) connects to an *MCP server* that exposes tools the agent can call.[a]

[a] https://en.wikipedia.org/wiki/Model_Context_Protocol

The relationship between A2A and MCP is complementary and worth stating clearly, because it is a common point of confusion: **A2A is agent-to-agent** (how two agents talk), whereas **MCP is agent-to-tool** (how one agent reaches an external capability such as a weather or payment service).

2.4 Secure transport: SLIM

SLIM (Secure Low-Latency Interactive Messaging)

A messaging framework that provides the *transport layer* for agent protocols such as A2A and MCP. While A2A defines what agents say, SLIM defines how those messages are delivered securely across distributed networks. It extends gRPC to support request/reply, fire-and-forget, publish/subscribe and group communication.[a]

[a] https://github.com/agntcy/slim

MLS (Messaging Layer Security)

An IETF standard for end-to-end encrypted group messaging. SLIM uses MLS in its session layer so that messages between agents are encrypted end to end, even across organizational boundaries.[a]

[a] https://en.wikipedia.org/wiki/Messaging_Layer_Security

SLIM is organised in two parts: a *data plane* that routes and delivers messages (with a session layer providing end-to-end encryption via MLS and reliable delivery), and a *control plane* that manages the routing infrastructure independently of application logic. In practice this means an operator can run a network of SLIM routing nodes, while agents use SLIM client libraries to communicate.

2.5 Agent description and discovery: OASF and the Agent Directory

OASF (Open Agent Schema Framework)

An extensible data model for describing an agent's attributes and ensuring unique identification. OASF can describe A2A agents and MCP servers. Agents publish an OASF description so that others can discover and understand them.[a]

[a] https://docs.agntcy.org/

Agent Directory

A registry where agents (or multi-agent applications) described in OASF are announced and can be discovered. Any organization can run its own directory and keep it in sync with others, forming a shared inventory — an "Internet of Agents".[a]

[a] https://docs.agntcy.org/dir/overview/

2.6 Identity and authorization: TBAC

TBAC (Tool-Based Access Control)

An authorization model that controls which agents may invoke which specific services or tools. Combined with an agent identity service, it answers the question "who is allowed to call what?".[a]

[a] https://docs.agntcy.org/identity/overview/

2.7 Telecom / service-orchestration standards (used by OpenSlice)

OSS (Operations Support System)

The software a network operator uses to provision, configure and manage network services. OpenSlice is an open-source OSS.[a]

[a] https://en.wikipedia.org/wiki/Operations_support_system

TMF Open APIs

A suite of standard REST APIs defined by the TM Forum for the telecom industry. For example, *TMF641 Service Ordering* standardises how a client orders a service. OpenSlice exposes TMF Open APIs to its clients.[a]

[a] https://www.tmforum.org/oda/open-apis/

ETSI NFV / SOL005

ETSI is the European Telecommunications Standards Institute. NFV (Network Functions Virtualization) replaces dedicated hardware with software running on general-purpose servers. *SOL005* is the ETSI interface specification through which an orchestrator drives an NFV Orchestrator (NFVO/MANO).[a]

[a] https://en.wikipedia.org/wiki/Network_function_virtualization

OAuth 2.0 / Keycloak

OAuth 2.0 is the industry-standard protocol for authorization. Keycloak is an open-source identity and access-management server that implements it; OpenSlice uses Keycloak for authentication.[a]

[a] https://en.wikipedia.org/wiki/OAuth

Apache ActiveMQ Artemis

A high-performance open-source message broker. It carries messages between software components using queues and publish/subscribe topics. OpenSlice uses Artemis as its internal message bus.[a]

[a] https://activemq.apache.org/components/artemis/

2.8 Semantic web standards (used by ontology approaches)

Ontology

In computer science, a formal, explicit specification of the concepts in a domain and the relationships between them — a shared vocabulary that lets different systems agree on what terms mean.[a]

[a] https://en.wikipedia.org/wiki/Ontology_(information_science)

Knowledge graph

A representation of knowledge as a graph of entities (nodes) and relationships (edges), often used together with ontologies to enable reasoning.[a]

[a] https://en.wikipedia.org/wiki/Knowledge_graph

3 Agntcy

3.1 Overview

Agntcy is an open framework for building an "Internet of Agents": a set of components and specifications that let independent agents discover one another, communicate securely, and be authorized. Rather than being a single application, it is a stack of building blocks: SLIM for transport (Section 2.4), A2A for agent communication (Section 2.2), OASF and the Agent Directory for description and discovery (Section 2.5), and an Agent Identity Service with TBAC for security (Section 2.6). MCP (Section 2.3) is used when an agent needs to reach external tools.

3.2 How Agntcy works, end to end

Figure 1 shows the general operation. The numbered steps describe a typical interaction for a reader unfamiliar with the system.

1. **Describe & register.** Each agent publishes an OASF description (an A2A "Agent Card") to the Agent Directory, declaring its identity and capabilities.

2. **Discover.** An agent that needs a capability queries the Agent Directory to find an agent that provides it — at runtime, rather than being hard-wired.

3. **Authorize.** The Agent Identity Service and TBAC determine whether the caller is permitted to invoke the target.

4. **Communicate.** The agents exchange A2A messages (JSON-RPC 2.0). These messages are carried over the SLIM transport, which encrypts them end to end (MLS).

5. **Use tools (optional).** If an agent needs external data or actions, it acts as an MCP client and calls an MCP server (e.g. a weather or payment service).

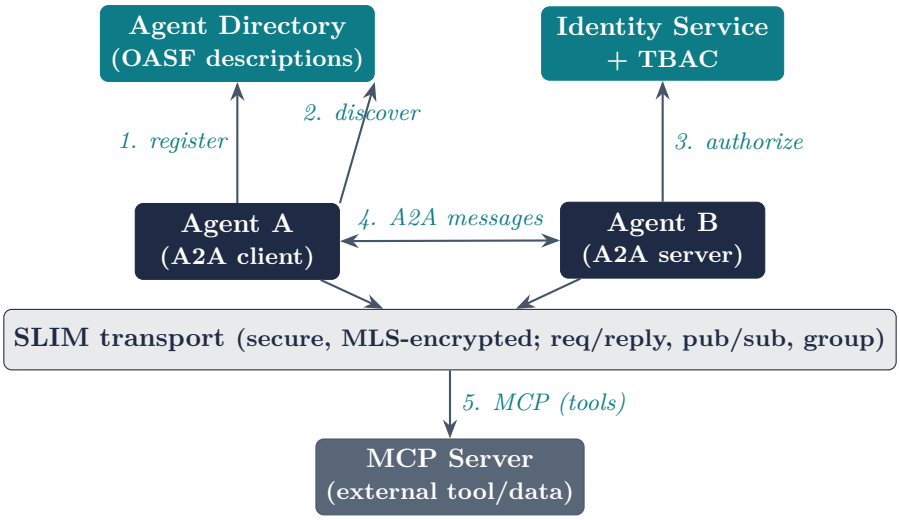


Figure 1: Agntcy general architecture and operation. Agents describe themselves to the Directory (1), discover each other (2), are authorized (3), then exchange A2A messages (4) carried securely over SLIM. Tool access (5) uses MCP.

3.3 The CoffeeAGNTCY reference applications

Agntcy ships two demonstration applications built around a fictional coffee company. Both use SLIM as the transport; they differ in scale and in the interaction patterns they exercise. They are described in detail below because they make the abstract protocols concrete.

3.3.1 Corto — the minimal two-agent flow

Corto (Italian for "short") is the intentionally minimal application: a two-agent coffee "sommelier". A supervisor/exchange agent acts as an A2A client; it forwards a user's question to a single grader (sommelier) agent acting as an A2A server, which returns a flavour-profile answer. It demonstrates the core A2A request/reply pattern over SLIM, with no tool use.



Figure 2: Corto operation. The user's question goes to the supervisor (A2A client), which sends an A2A request over SLIM to the sommelier (A2A server); the reply returns along the same path. A single request/reply exchange, no MCP tools.

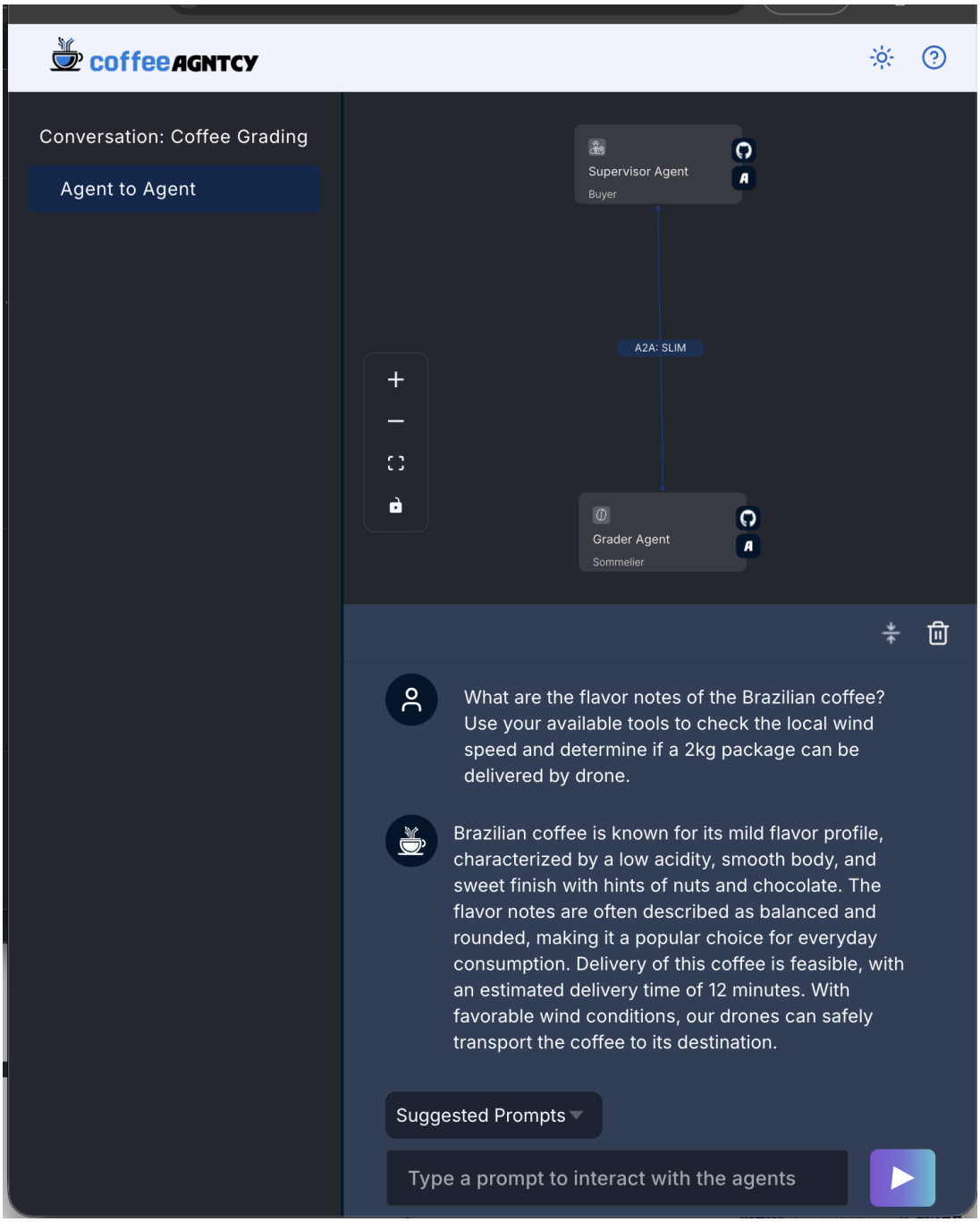


Figure 3: Corto running — supervisor and grader agents, A2A:SLIM link visible.

3.3.2 Lungo — the larger multi-agent flow

Lungo (Italian for "long") expands the demonstration into a larger ecosystem. A LangGraph-orchestrated supervisor coordinates several agents in a supervisor–worker pattern, and some agents use MCP to reach external tools. Here SLIM is exercised more fully: in addition to request/reply it uses publish/subscribe (one-to-many broadcasts) and group communication (several agents sharing a session), with NATS available as an alternative transport for pub/sub.

LangGraph

A framework for building stateful, multi-step applications with language models, structured as a graph of steps. In Lungo it provides the supervisor–worker orchestration logic.[a]

[a] https://www.langchain.com/langgraph

Supervisor–worker pattern

An orchestration pattern in which one "supervisor" agent decomposes a task and delegates sub-tasks to "worker" agents, then aggregates their results.[a]

[a] https://en.wikipedia.org/wiki/Master%E2%80%93slave_(technology)

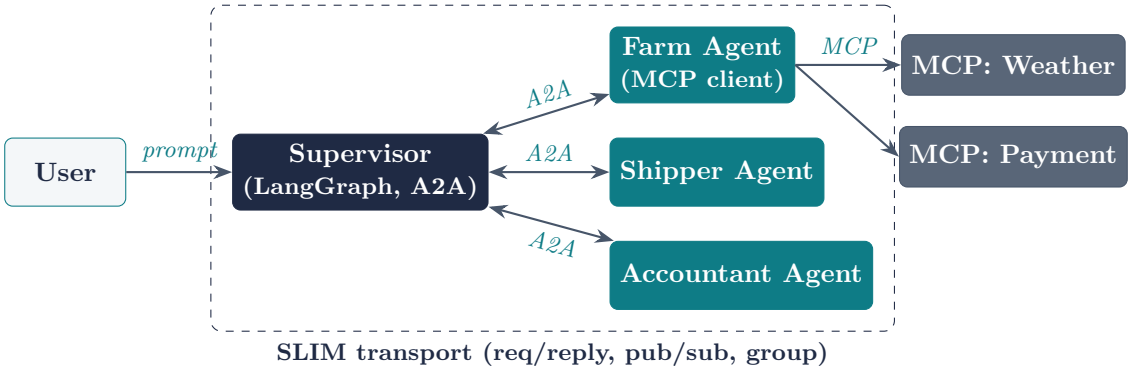


Figure 4: Lungo operation. A LangGraph supervisor coordinates several worker agents over A2A (carried on SLIM, using request/reply, publish/subscribe and group patterns). The farm agent additionally acts as an MCP client to reach external Weather and Payment tools.

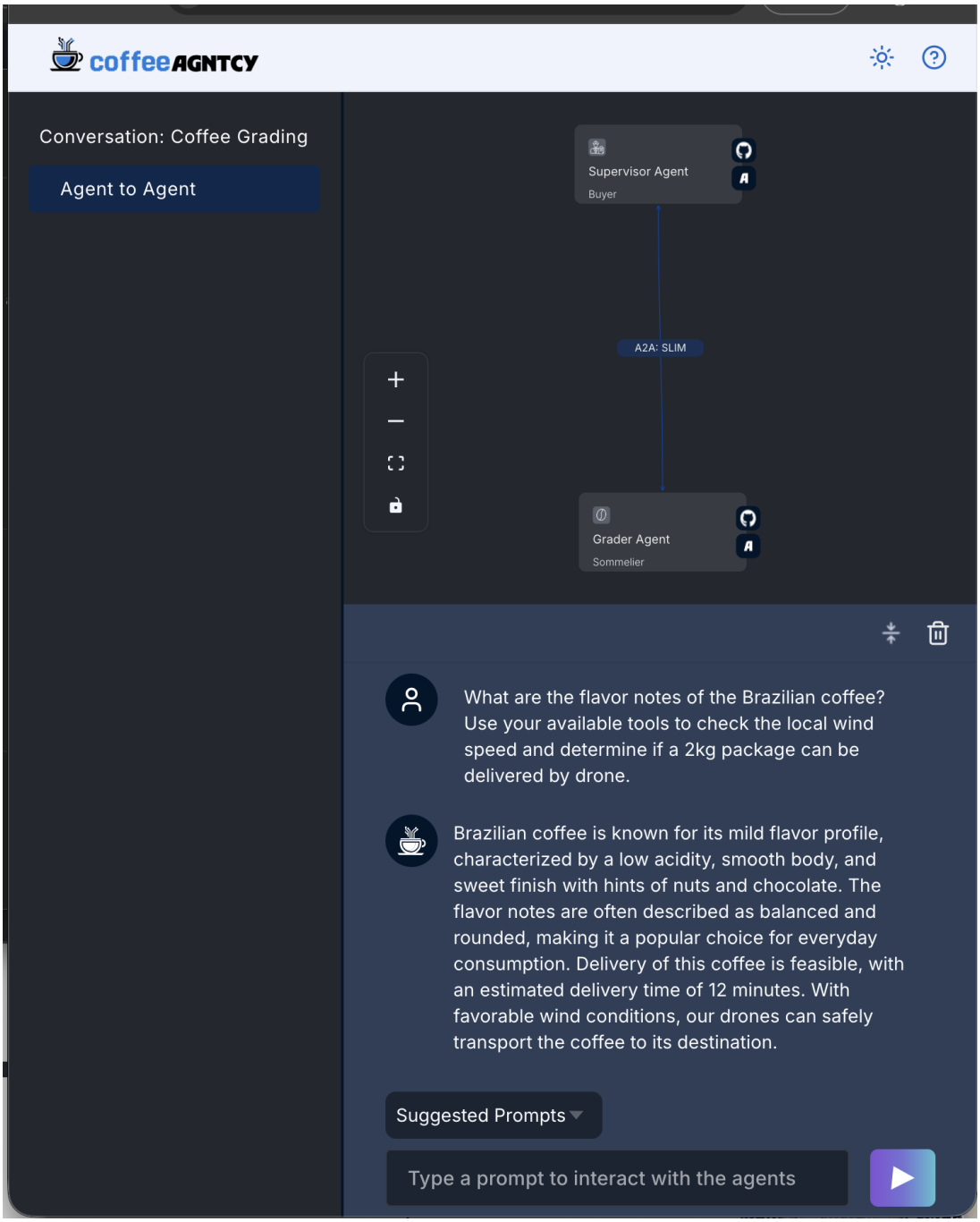


Figure 5: Lungo running — multi-agent order flow with the message trail.

3.3.3 Corto versus Lungo

The two applications are best understood as the same architecture at two scales.

	Corto	Lungo
Agents	Two	Several (supervisor + workers)
Complexity	Minimal / entry-level	Advanced multi-agent system
Orchestration	Direct A2A request/reply	Supervisor–worker (LangGraph)
SLIM patterns	Request/reply	Request/reply, pub/sub, group
MCP tools	Not used	Used (e.g. weather, payment)
Alt. transport	—	NATS (for pub/sub)
Purpose	Show the core A2A pattern	Show a full agentic ecosystem

Table 2: Corto and Lungo compared. Both run on SLIM; Corto isolates the core request/reply pattern, Lungo exercises the richer messaging patterns and adds MCP tool use.

3.4 Agntcy in the six-layer framework

Communication, discovery, orchestration and security are directly addressed by Agntcy's components (SLIM, the Directory/OASF, the supervisor–worker pattern, and Identity/TBAC respectively). The semantics and reasoning layers are not the focus of the framework: agents exchange messages and can call tools, but Agntcy itself does not define a shared ontology or an inference mechanism. This is a description of scope, not a deficiency — Agntcy is positioned as a communication and identity fabric.

4 OpenSlice

4.1 Overview

OpenSlice, hosted by ETSI, is an open-source Operations Support System (OSS) for delivering "Network as a Service". Where Agntcy targets direct agent-to-agent messaging, OpenSlice targets *service orchestration*: a client orders a service through standard APIs, and OpenSlice provisions and manages it across the underlying infrastructure. It is built as a set of microservices.

Microservice architecture

A way of building an application as a collection of small, independently deployable services that communicate over a network, rather than as one monolith.[a]

[a] https://en.wikipedia.org/wiki/Microservices

4.2 How OpenSlice works, end to end

A request is a *service order* that flows through the system from the client to the underlying network, as shown in Figure 6.

1. A third-party application or the portal submits a service order through the **TMF Open APIs** (e.g. TMF641).
2. The request passes an **API gateway** and is authenticated via **Keycloak** (OAuth 2.0).
3. It reaches the **TMF API** microservices (Service Catalog, Service Ordering, . . .).
4. Internally, the microservices communicate over the **Artemis** message bus using queues and publish/subscribe topics.

5. The **OSOM** orchestrator instantiates the service specification and drives its lifecycle, translating the order into network-service requests to an **NFVO/MANO** via ETSI **SOL005**.

OSOM (OpenSlice Service Orchestrator)

The OpenSlice component that takes a service order and instantiates the corresponding service specification, coordinating the steps needed to fulfil and manage it over its lifecycle.[a]

[a] https://openslice.io/

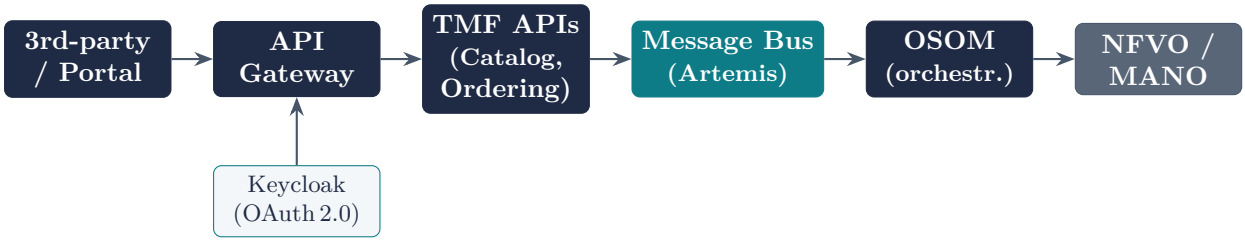


Figure 6: OpenSlice operation. A service order flows left to right through standard TMF Open APIs; microservices coordinate over the Artemis message bus; OSOM orchestrates and drives the underlying NFVO/MANO via ETSI SOL005. Keycloak provides OAuth 2.0 authentication.

4.3 Deployment status

For this study, OpenSlice was deployed locally using Docker. The core services started successfully — Keycloak, MySQL, the TMF/Portal APIs, the Artemis broker and OSOM. The broker console and the home page were reachable. The portal login page returned a "Not Found" response, attributable to a front-end configuration issue. The deployment was nonetheless sufficient to study the architecture and the internal communication mechanisms described above.

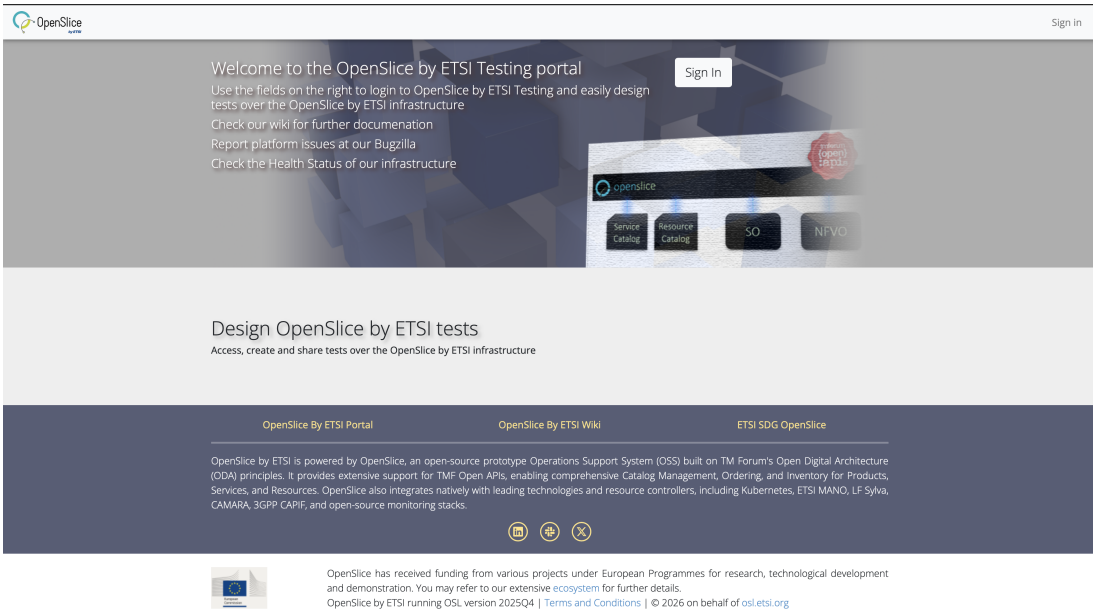


Figure 7: OpenSlice home page (Welcome to OpenSlice by ETSI).

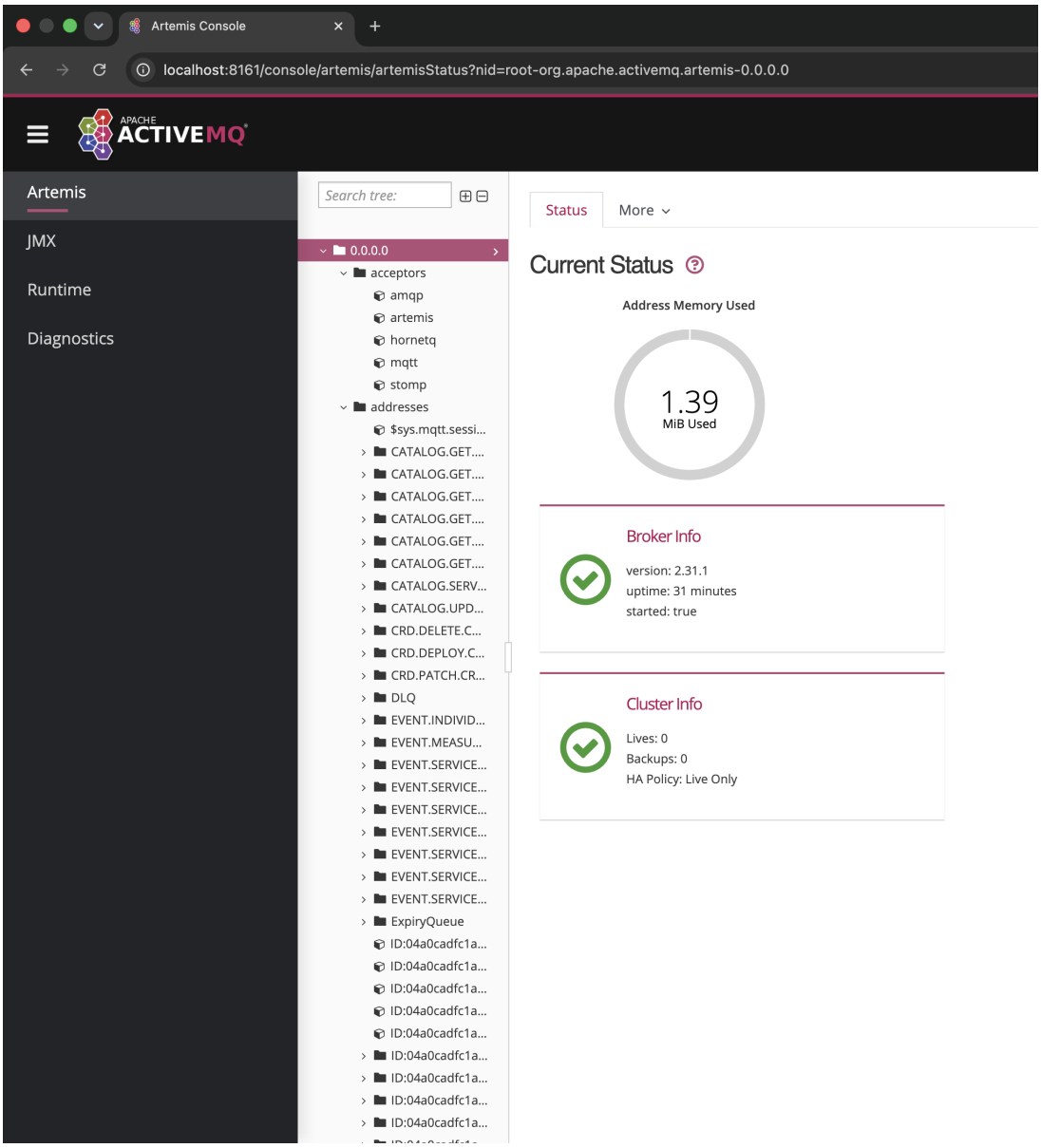


Figure 8: ActiveMQ Artemis management console — broker status (version, uptime).

4.4 OpenSlice in the six-layer framework

OpenSlice strongly addresses orchestration and, through TMF APIs and SOL005, interoperability at the service level; security is provided via Keycloak/OAuth 2.0; communication between components is provided by the Artemis bus. Like the agent-communication platforms, it does not provide a shared semantic model or reasoning over the exchanged data — it orchestrates services rather than reasoning about meaning.

5 W3C Agent Protocol, MCP and GITER

5.1 W3C / A2A agent protocols

A2A (Section 2.2) is the most developed agent-to-agent communication standard. It uses established web standards (HTTP, JSON-RPC 2.0, SSE), defines discovery through the Agent Card, and is under neutral governance at the Linux Foundation. Its purpose is *horizontal interoperability*: letting agents built by different vendors talk to each other without bespoke point-to-point

integrations.

5.2 MCP

MCP (Section 2.3) standardises the vertical direction — connecting an agent to tools and data. It is complementary to A2A: A2A handles agent-to-agent orchestration, MCP handles an agent's access to external capabilities.

5.3 GITER

GITER

A 3GPP-related inter-agent communication framework associated with the telecom/OpenSlice ecosystem. Like the other standards in this section, its concern is communication and interoperability between agents rather than semantics or reasoning.[a]

[a] See 3GPP documentation; https://www.3gpp.org/

5.4 Common observation

All three standards in this section address how agents (or agents and tools) *communicate* and *interoperate*. None of them, on its own, defines a shared *meaning* for the exchanged data or a mechanism to *reason* over it. That concern belongs to the semantic layer, discussed next.

6 Semantic Reasoning Approaches

6.1 Communication is not understanding

A central observation of this survey is that successful message delivery does not imply shared understanding. Two agents can exchange syntactically perfect messages and still not agree on what the content means. If agent A emits `AV` intending "autonomous vehicle" and agent B has no matching concept, the message was communicated but not understood.



Figure 9: The semantic gap: identical bytes do not guarantee identical meaning.

6.2 Ontologies as shared meaning

The semantic-web approach to this problem is to give agents a shared, machine-readable vocabulary — an *ontology* (Section 6). With a common ontology, agents interpret the same term in the same way, and a reasoning engine can derive new facts from existing ones. In the autonomous-agent context, the relevant building blocks discussed in the literature this project drew on include a machine-learning ontology (*MLOnto*), an AI knowledge-representation model (*AIKR*), and an agent ontology providing a shared agent vocabulary. (These are referenced here as the semantic components relevant to the domain; their detailed formal specification is beyond the scope of this survey.)

6.3 Do agent ecosystems require an LLM?

A frequent question is whether a large language model is necessary. Based on the survey, the answer is that it is not required for the lower layers: communication, discovery and orchestration are provided by protocols (A2A, SLIM, the Directory), and security by identity and access

control. Semantics does not necessarily require an LLM, and reasoning can be provided by ontologies, rule engines, planning systems or knowledge graphs. An LLM is therefore one possible component — useful for flexible natural-language behaviour — rather than a mandatory part of the ecosystem.

LLM (Large Language Model)

A machine-learning model trained on large amounts of text to generate and interpret natural language. In agent systems it is often the "brain" behind an agent, but it is not required for the communication and coordination layers.[a]

[a] https://en.wikipedia.org/wiki/Large_language_model

7 Comparative Discussion

The surveyed technologies occupy different positions in the landscape and are, to a large extent, complementary rather than competing. Table 3 summarises which concerns each was designed to address. The entries describe *design scope*, not a quality ranking.

Technology	Comm.	Discovery	Orchestr.	Semantics/Reasoning
Agntcy (SLIM+A2A)	core	core	yes	not in scope
MCP	tool link	—	—	not in scope
W3C / A2A	core	via Agent Card	partial	not in scope
OpenSlice	internal bus	service-level	core	not in scope
GITER	yes	—	—	not in scope
Ontologies	—	—	—	core

Table 3: Design scope of each technology against the six-layer framework. "Not in scope" means the technology was not designed to address that layer, not that it performs poorly.

Key differences in one view.

- **Agntcy vs OpenSlice:** direct agent-to-agent messaging (A2A over SLIM) versus service orchestration via standard telecom APIs and an internal message bus.
- **A2A vs MCP:** agent-to-agent versus agent-to-tool; complementary directions.
- **Corto vs Lungo:** the same Agntcy architecture at minimal vs full scale (see Figures 2 and 4).
- **Communication standards vs ontologies:** moving messages versus sharing meaning.

8 Gap Analysis and Conclusion

8.1 Findings

- Several mature technologies already address communication and interoperability between agents and services (Agntcy/SLIM/A2A, MCP, W3C/A2A, OpenSlice, GITER).
- These technologies are largely complementary: they cover different layers of the six-layer framework.

- Across all of them, the *semantic* and *reasoning* layers are comparatively underdeveloped. Communication standards move messages reliably but do not, by themselves, provide shared meaning or inference.

8.2 Gap

The principal gap identified by this survey is therefore not in communication but in *understanding*: the surveyed platforms ensure that agents can exchange messages and invoke tools, but shared semantics and reasoning are not provided by the communication layer itself. This is consistent with the design scope of each technology — the communication and identity platforms were not intended to be reasoning layers. Closing this gap (for example by combining a communication fabric with an ontology-based reasoning layer) is a direction for future work rather than a conclusion of this descriptive survey.

8.3 Summary

This report has surveyed the current state of the art of agentic AI platforms for autonomous multi-agent ecosystems, defined the relevant protocols and standards, and described in detail how Agntcy (including its Corto and Lungo applications) and OpenSlice operate. The consistent observation across the landscape is a well-developed communication and interoperability layer alongside a comparatively thin semantic-reasoning layer.

Appendix: Index of defined terms

Autonomous agent • Multi-agent system • HTTP/HTTPS • JSON/JSON-RPC • gRPC • SSE • Publish/subscribe • A2A • MCP • SLIM • MLS • OASF • Agent Directory • TBAC • OSS • TMF Open APIs • ETSI NFV/SOL005 • OAuth/Keycloak • Artemis • Ontology • Knowledge graph • LangGraph • Supervisor–worker • Microservice • OSOM • LLM.